

A TALK FOR FIRST & SECOND-YEAR CSE

# Why fresh minds may win the AI era.

AI makes coding more accessible — which is exactly why fundamentals, judgment, and curiosity become more valuable.

VIBHOR

VP ENGINEERING · CARESMARTZ360

CHITKARA '08

# Judgment.

THE PART YOU CANNOT OUTSOURCE

# Same student energy, very different tools

2008



GRADUATION • A BEGINNER WITH  
QUESTIONS

SAME CURIOSITY  
→  
FAR MORE LEVERAGE

2026



TODAY • SAME QUESTIONS, SHARPER  
TOOLS

A BETTER WAY TO READ THIS MOMENT

AI makes the *first draft* easier. That makes *thinking* the edge.

SURFACE

**Code appears faster.**

Boilerplate, scaffolding, and first drafts arrive in minutes.

DEEPER

**Thinking decides what deserves to exist.**

What to build, why, and how to know it's right — that's still on you.

PERSONAL NOTE • 2008

# My final-year project was an AI chatbot.

Same building, same curiosity. Just much harder to actually build.

What changed most by 2026 is not ambition — it's the cost of trying ideas.



IBOT • MAJOR PROJECT • 2008 • [PROJECT REPORT](#)



EIGHTEEN YEARS LATER

**What changed.  
What endured.**



# Changed

The cost of going from idea to working software dropped sharply.

- 1 Boilerplate is essentially free.
- 2 Prototypes ship in hours, not weeks.
- 3 AI assists across debugging, testing, review, and docs.
- 4 Trying an ambitious idea is dramatically easier.



# Endured

The things that decide whether software is actually good barely moved.

- 01 Correctness still depends on mental models.
- 02 Users still depend on the choices we make.
- 03 Systems still hide assumptions until they break.
- 04 Curiosity and judgment still compound over time.



FROM THE FIELD

**Beginners are  
already shipping.**

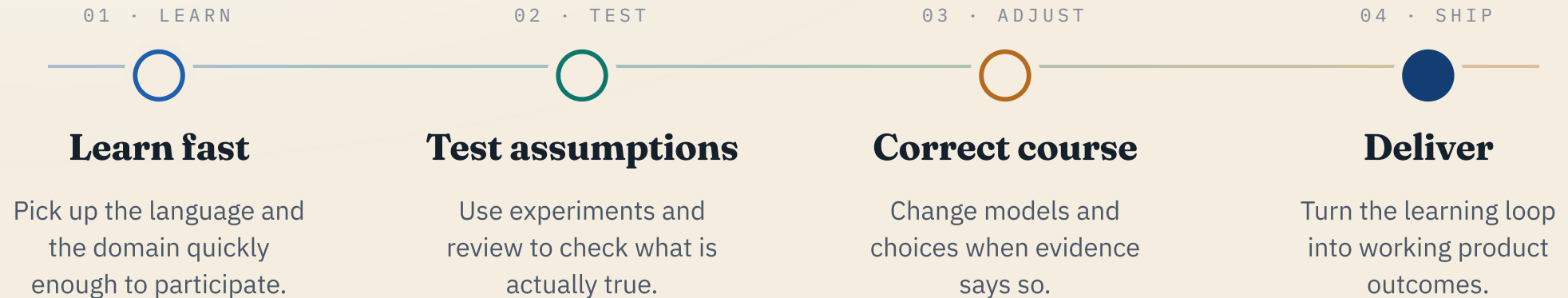


**Freshers  
contributed to *real  
product work* when  
they had context,  
trust, and useful  
tools.**

- They learned fast when the work was framed well.
- They asked sharper questions as context grew.
- They improved quickly with trust and tooling in place.

# Readiness grew through movement, not credentials

When the path changed mid-project, the team learned fast and shipped — by owning the problem.



# Engineering work is now humans + AI agents

AI extends across debugging, testing, review, documentation, and communication. Judgment stays with the human.



# What stays human-owned

The better the human judgment, the better the AI outcome.

01

## Correctness

Deciding what "right" means, and proving the system meets it.

02

## Privacy

What data is collected, what's shared, and with whom.

03

## Bias & safety

Who can be harmed, and what guardrails actually hold.

04

## Consequences

Owning what the system does once it lives in the world.



THE DEEPER OPPORTUNITY

# **Faster tools make fundamentals more valuable.**

When implementation gets cheap, understanding is  
the part you cannot outsource.

# Classroom topics become product superpowers

DSA

## Data structures

Scheduling conflicts and indexes are interval problems in disguise.

DBMS

## Databases

Indexes, isolation, and version compatibility decide real outcomes.

OS

## Operating systems

Processes, memory, and isolation become concrete once you build one.

NET

## Networks

Reliability, ordering, and retries are lived trade-offs in production.

QA

## Testing

Testing is structured doubt — equally useful for humans and AI.

ARCH

## Architecture

Alternatives, boundaries, and operational cost remain the central call.

REAL EXAMPLE • 1 OF 2

# A caregiver calendar is also an algorithm problem.

Two visits overlap. Suddenly intervals, unions, and recurrence stop being abstract.

Abstraction is the tool that turns rich product reality into something solvable.



REAL EXAMPLE • 2 OF 2

## SQL behavior reveals what your code is really standing on.

Same query, different version, different result. The stack underneath is doing real work.

AI helps more when the engineer understands the ground truth.

```
mysql 5.7 != mysql 8.0
```

|                  |           |
|------------------|-----------|
| product query    | YOUR CODE |
| driver behavior  | LIBRARY   |
| database version | ENGINE    |
| environment      | INFRA     |





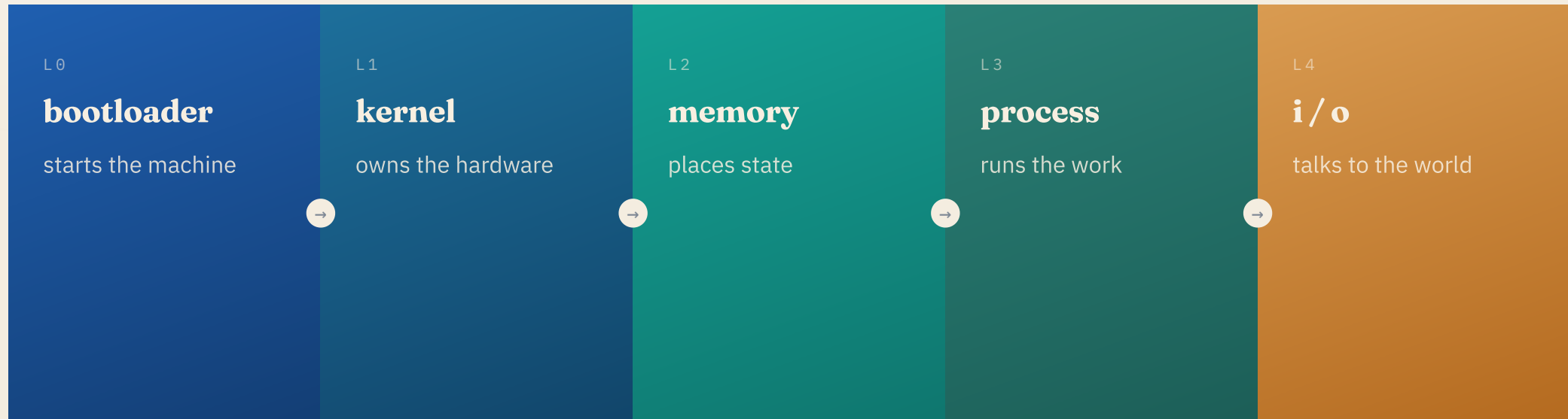
2026 ADVANTAGE

# Build the lesson while you learn it.

For the first time, ambitious students can experience systems thinking directly — instead of waiting for "later".

# If you're learning operating systems, build a tiny world

Vibe-code a hello-world flavor of Linux. Let boot, memory, processes, and I/O become felt experiences.



IN 2026, THIS IS PRACTICAL ENOUGH TO TRY WHILE YOU LEARN.

# If you're learning networks, send real packets

Build a tiny app. Send the same data over both TCP and UDP. Watch the trade-offs become visible.

**TCP**

ORDERED ·  
ACKNOWLEDGED



→ RECEIVER

**UDP**

FAST · NO  
GUARANTEES



→ RECEIVER

TCP

Reliability and ordering, paid for in latency.

UDP

Speed and freshness, paid for in lost packets.



# How to build your beginner's advantage now

Practice that compounds — through fundamentals, AI, and your own taste.

- 1 Start from **first principles** before you reach for a prompt.
- 2 Build **small, personal, useful systems** while you study.
- 3 Use AI as a **thought partner** — keep your reasoning active.
- 4 Write tests, explain trade-offs, **keep your evidence**.
- 5 Ask **better questions** than your peers.

BUILDER'S NOTE

# Fresh minds are arriving at exactly the right time.

The field needs beginners who can think clearly, build courageously, and turn faster tools into trustworthy systems.

**Thinkers** | **Builders** | **Responsible engineers**

# The honest ending: jobs *will* change

Some roles disappear. New ones appear. Many existing roles change. People with judgment, curiosity, and first-principles thinking rarely go out of fashion.

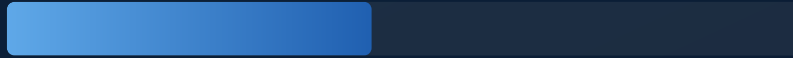
**170M**  
JOBS CREATED



**92M**  
JOBS DISPLACED



**+78M**  
NET GROWTH



**22%**

OF JOBS DISRUPTED BY  
2030

SOURCE · WORLD ECONOMIC FORUM · FUTURE OF JOBS REPORT 2025